

Debugging CSS with DevTools

In the last lesson, you learned that CSS has rules for deciding which declaration wins.

That knowledge is useful, but you do not have to solve every CSS problem in your head. The browser can show you what it actually did. That is what DevTools is for.

This lesson is about one practical habit: when CSS surprises you, inspect the element before guessing.

Start with a page that has CSS problems

Use a two-file page with a few CSS problems already built in: `index.html` and `style.css` .

Start with this `index.html` file:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>DevTools CSS practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Debug a styled card</h1>
        <p class="lead">DevTools helps you see which CSS matched, which CSS lost, and wh<
      </section>

      <article class="card featured-card" id="starter-plan">
        <div class="tag-row">
          <span>Popular</span>
          <span>Beginner friendly</span>
        </div>

        <h2>Starter plan</h2>
        <p class="card-text">A small card with a few styles that need debugging.</p>
        <p class="notice">Only a few seats are available this week.</p>
      </article>
    </main>
  </body>
</html>
```

Then add this `style.css` file with the intentional problems:



CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 760px;
  margin: 40px auto;
}

.intro,
.card {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.intro {
  margin-bottom: 24px;
}

.eyebrow {
  color: #2563eb;
  font-weight: 700;
}

.lead,
.card-text {
  color: #4b5563;
  line-height: 1.6;
}
```

< Previous Lesson

Next Lesson >

```
,

.tag-row {
  gap: 8px;
  margin-bottom: 16px;
}

.tag-row span {
  background-color: #eff6ff;
  color: #1d4ed8;
  padding: 6px 10px;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 700;
}

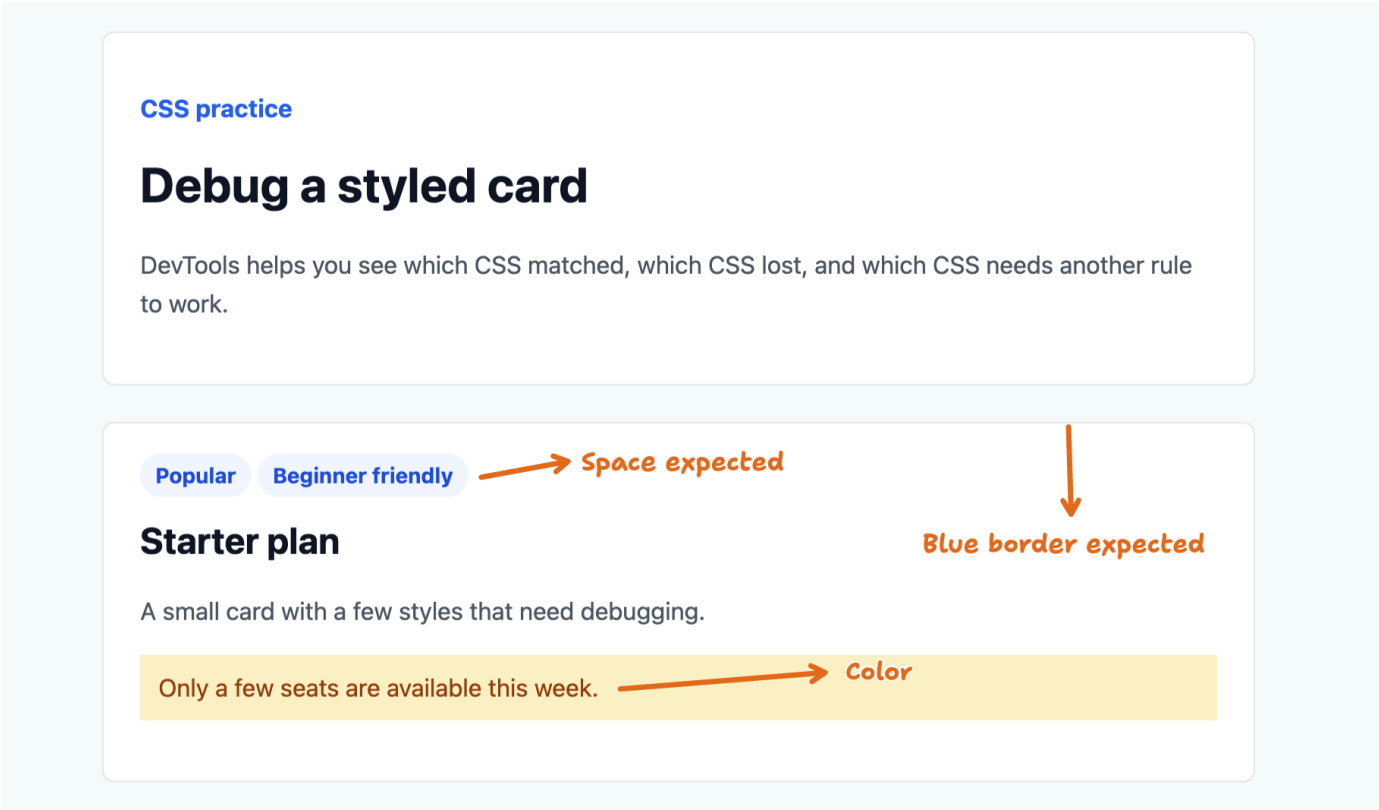
.notice {
  color: #047857;
  background-color: #fef3c7;
  padding: 12px;
  margin-top: 16px;
}

.card .notice {
  color: #92400e;
}
```

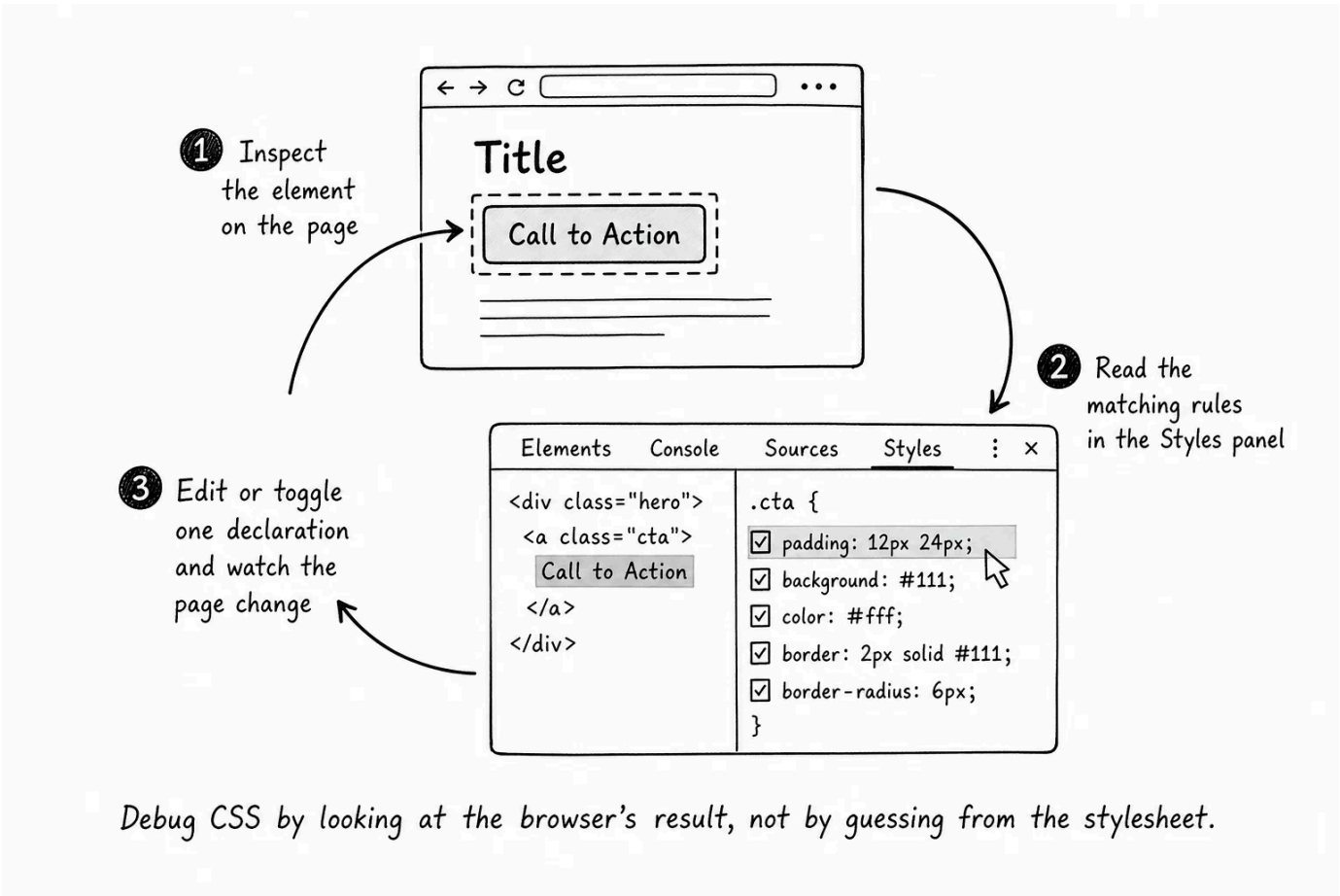
Open the page in a browser.

The CSS file creates three small problems on purpose:

- The featured card border is not blue.
- The notice text color may not be the one you expected from `.notice`.
- The tag pills do not have a visible gap between them.

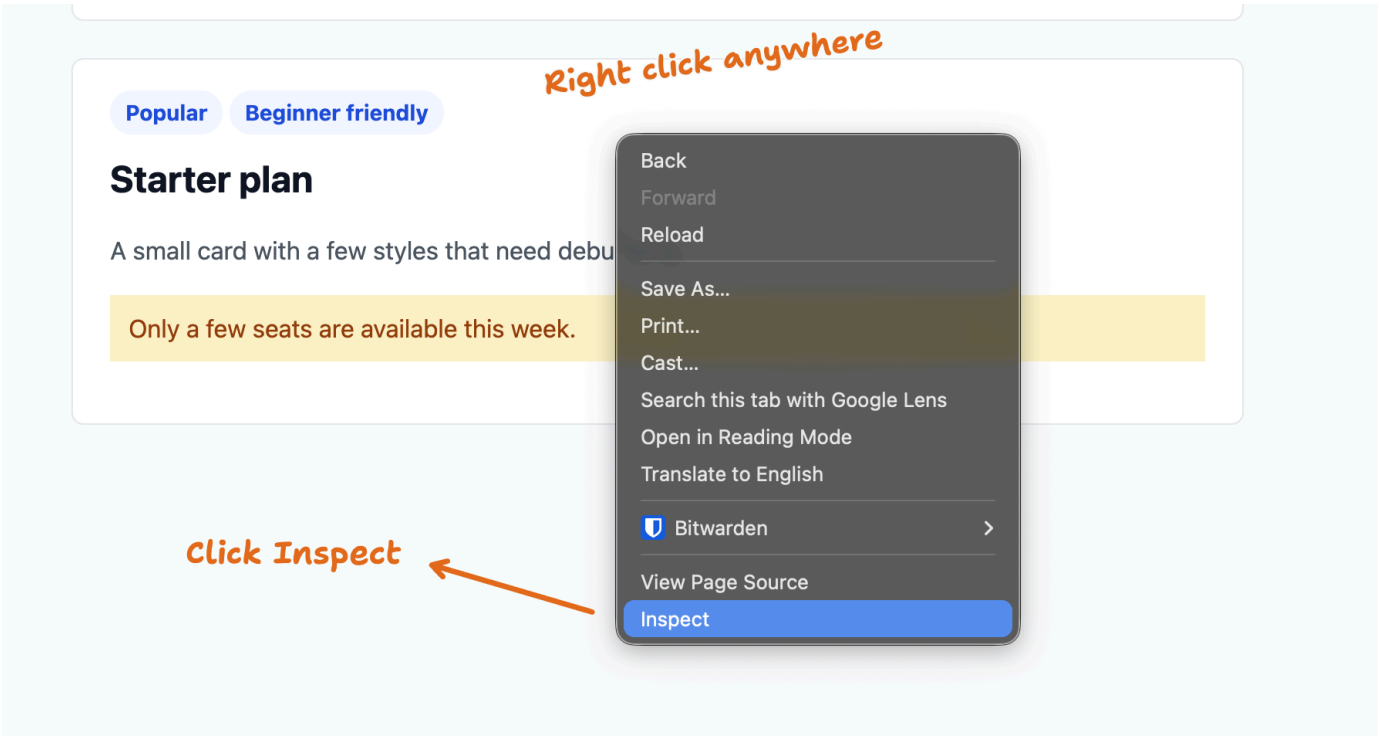


Do not fix them yet; inspect them first.

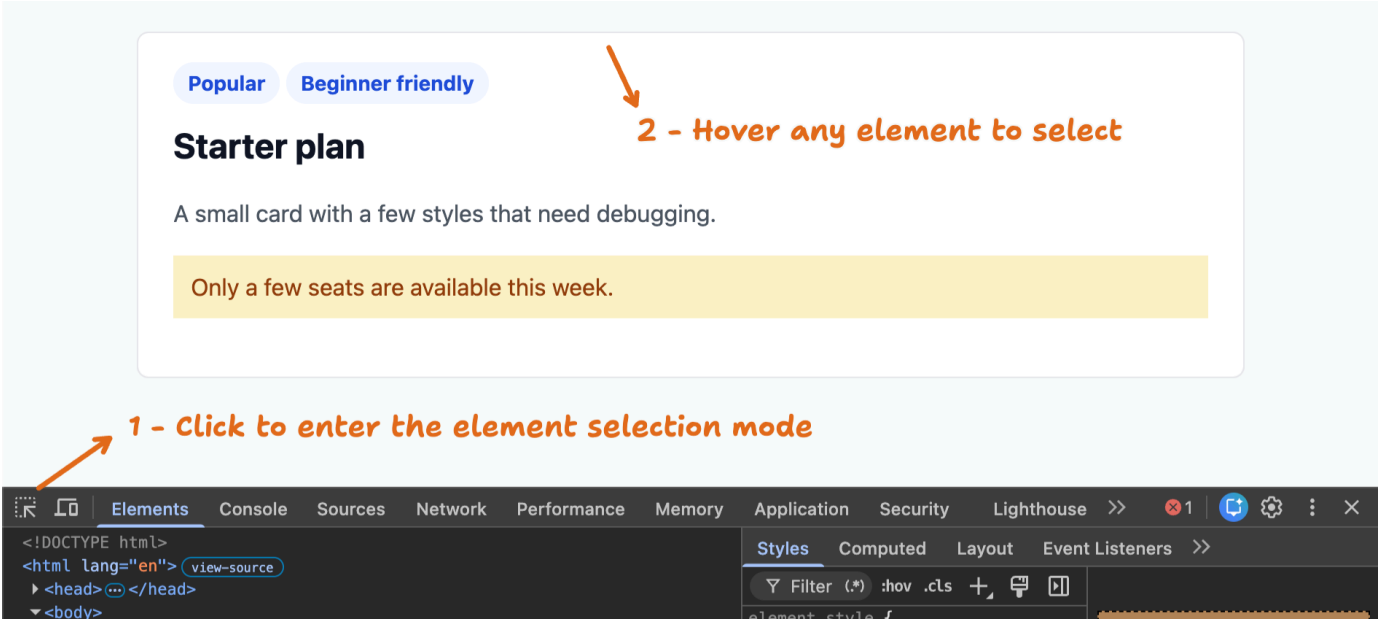


Inspect the element

Open DevTools and inspect the featured card.



In most browsers, you can right-click the card and choose **Inspect** . You can also open DevTools first, then use the element picker to click the card on the page.



For CSS debugging, the first question is simple: am I inspecting the right element?

If you inspect the card, the selected HTML should look like this:

```
html

<article class="card featured-card" id="starter-plan">
```

That selected element is the thing DevTools will explain.

For the first bug, two parts of DevTools matter most: **Elements** shows the selected HTML, and **Styles** shows the CSS rules that match it. Later in the lesson, you will also use **Computed** and the box model diagram.

DevTools may also show browser default styles, often labeled **user agent stylesheet** . These are styles built into the browser. You will see them most often on links, form elements, and headings, and they follow the same cascade rules as your CSS.

💡 **Inspect the exact element** - If a style affects a paragraph, inspect the paragraph. If it affects the card border, inspect the card. CSS debugging gets confusing when you are looking at a parent or child by accident.

Check whether the selector matched

The card is supposed to have a blue border because the HTML has `featured-card`:

html

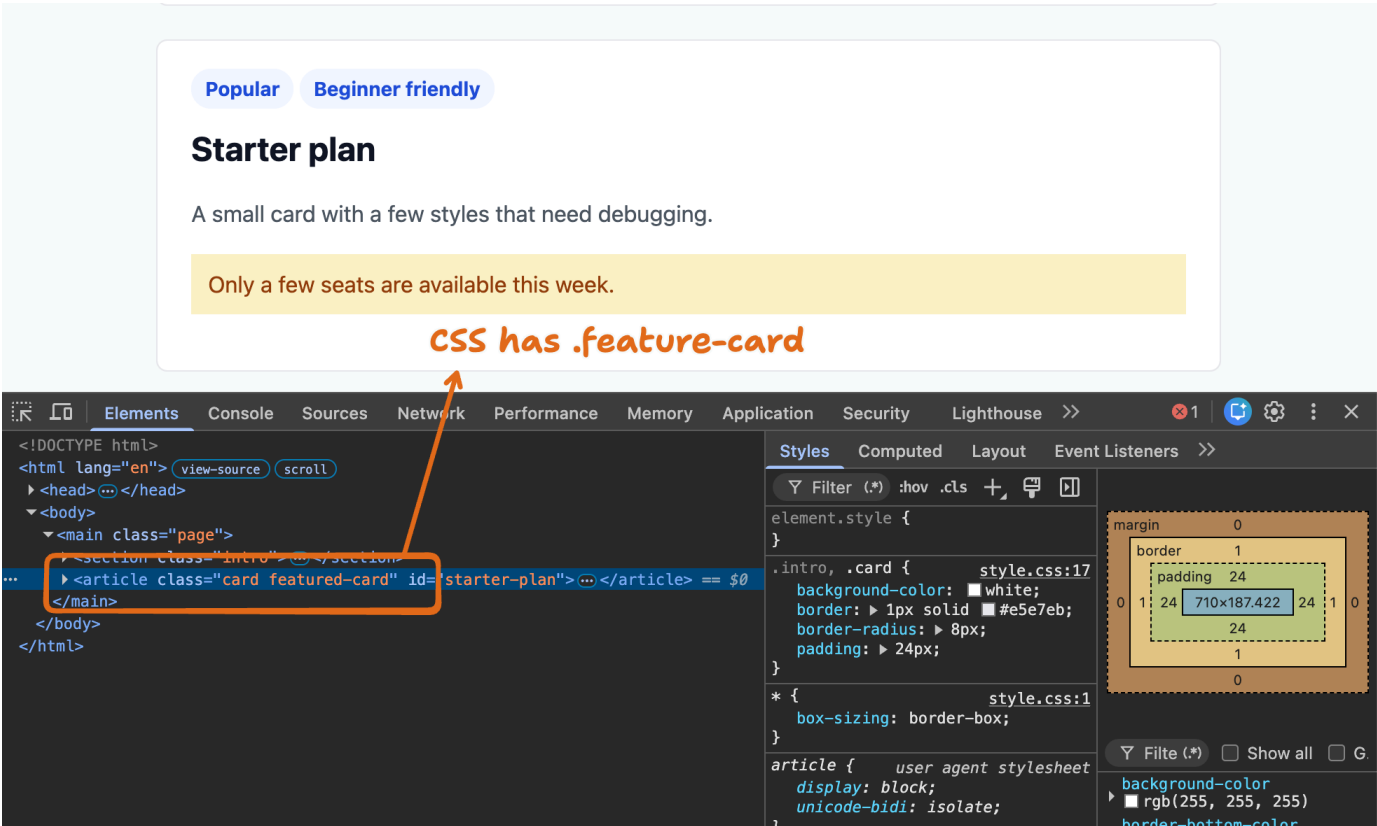
```
<article class="card featured-card" id="starter-plan">
```

But the CSS is looking for a different class:

CSS

```
.feature-card {  
  border-color: #2563eb;  
}
```

Inspect the card and look in the `Styles` panel. You should see the `.card` rule, but you will not see `.feature-card` as a matching rule for this element.



That is the clue: the selector does not match because the class names are different.

- HTML: `featured-card`
- CSS: `.feature-card`

Fix the selector by matching the HTML class name:

CSS

```
.featured-card {  
  border-color: #2563eb;  
}
```

Reload the page and the card border should become blue.

When a rule does not appear in DevTools at all, the selector probably did not match the element.

Invalid CSS is a different clue. If a rule appears but a value is invalid, DevTools may cross out that declaration or show a warning icon. That is not the same as a valid declaration losing in the cascade. The browser usually ignores invalid CSS silently instead of showing an error message.

Check whether the declaration lost

Next, inspect the notice:

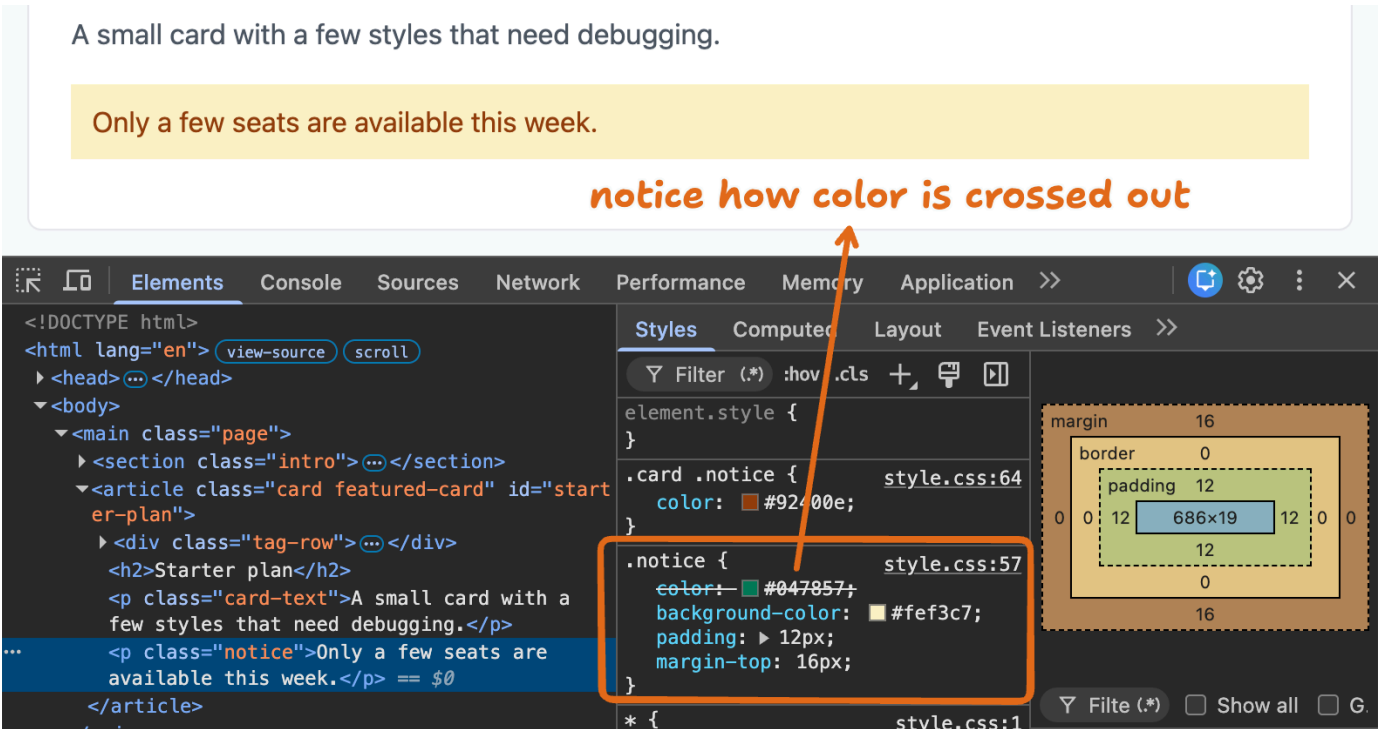
```
html  
  
<p class="notice">Only a few seats are available this week.</p>
```

Two rules set its text color:

```
CSS  
  
.notice {  
  color: #047857;  
}  
  
.card .notice {  
  color: #92400e;  
}
```

The `.notice` selector matches, and the `.card .notice` selector also matches because the notice sits inside `.card`.

In DevTools, you may see the `.notice` color crossed out and the `.card .notice` color active.



That means the selector matched, but the declaration lost.

This is different from the previous bug. `.feature-card` did not match at all. `.notice` matches, but its `color` declaration loses to a more specific selector.

This is the cascade from lesson 7 in action: both selectors match, but the more specific selector wins.

Check whether the property needs setup

The tag row has this CSS:

```
CSS

.tag-row {
  gap: 8px;
  margin-bottom: 16px;
}
```

The `gap` declaration is valid, but it does not create visible space yet.

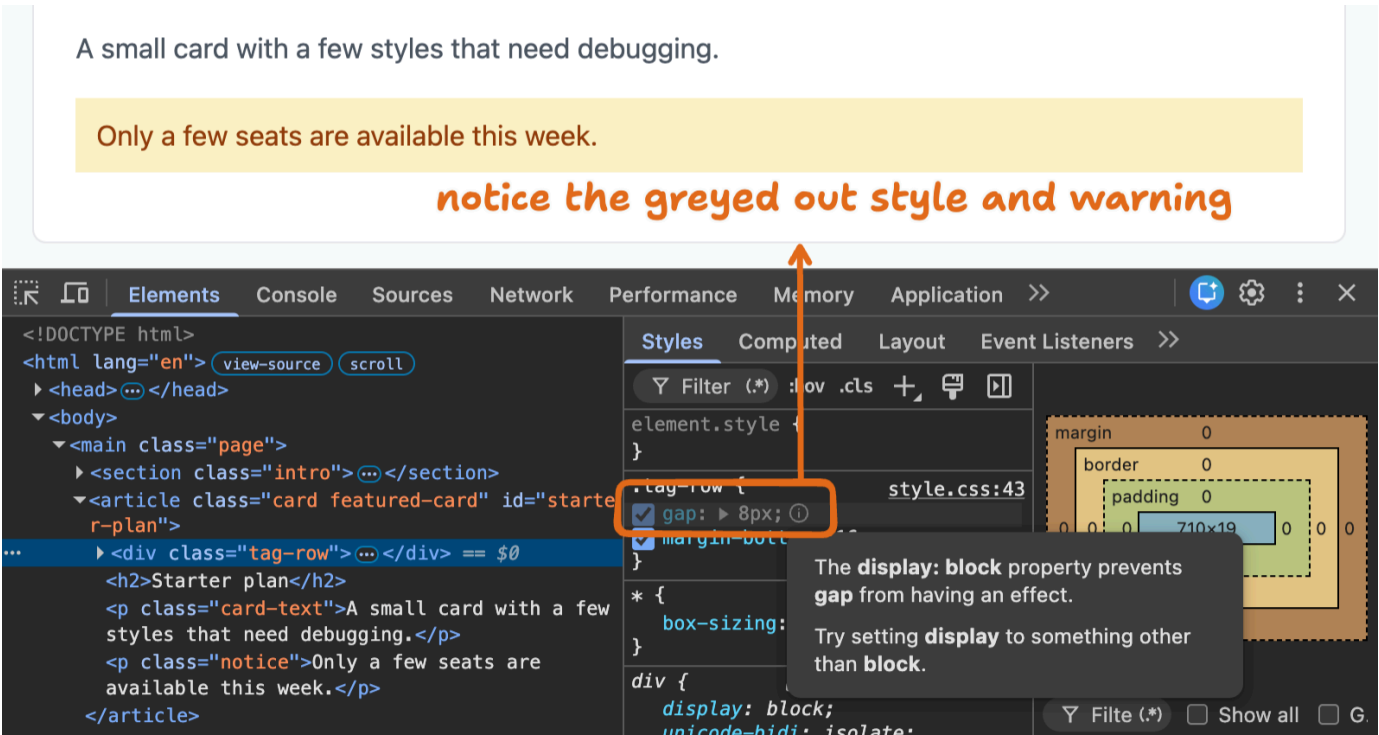
The reason is that `gap` needs a layout mode like flex or grid on the parent. You saw this in the spacing lesson.

Fix the parent by adding `display: flex` :

```
CSS

.tag-row {
  display: flex;
  gap: 8px;
  margin-bottom: 16px;
}
```

Reload the page and the tag pills should now have space between them.



This is another debugging category: the selector matched and the declaration may not be crossed out, but the property still needs the right surrounding

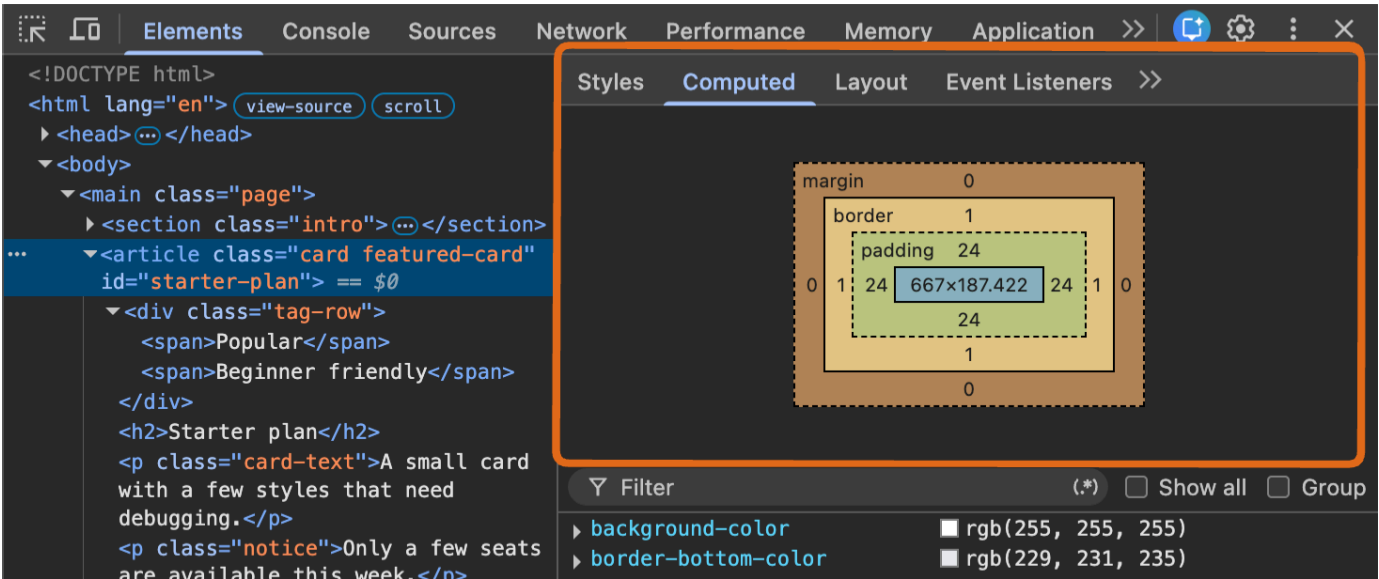
CSS to have a visible effect.

⚠ **Not every CSS surprise is the cascade** - If a declaration is not crossed out but still does not seem to change the page, ask whether the property needs another setup rule. `gap` needs flex or grid. Some layout properties only work in certain contexts.

Use the box model diagram

DevTools can also show the box model.

Inspect the card and look for the box model diagram in the **Computed** panel or layout area. You should see content in the middle, then padding, border, and margin around it.



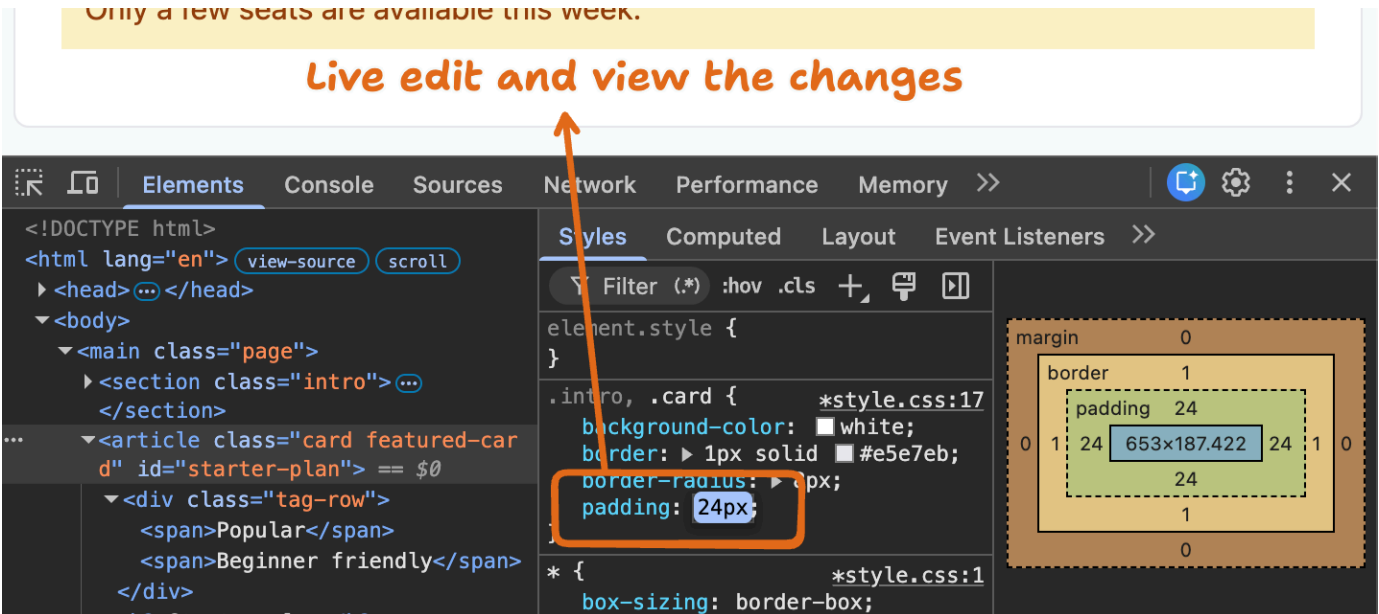
Find these values from the CSS:

```
CSS

.card {
  border: 1px solid #e5e7eb;
  padding: 24px;
}
```

The diagram should show **24px** padding and a **1px** border.

Now temporarily change `padding: 24px;` to `padding: 8px;` inside DevTools, and the card should update immediately.



This live edit does not save to your file. It is a safe way to test a value before changing `style.css` .

 **Use DevTools as a scratchpad** - When you are unsure about a spacing, color, or size value, try it in DevTools first. Once it looks right, copy the final value into your CSS file.

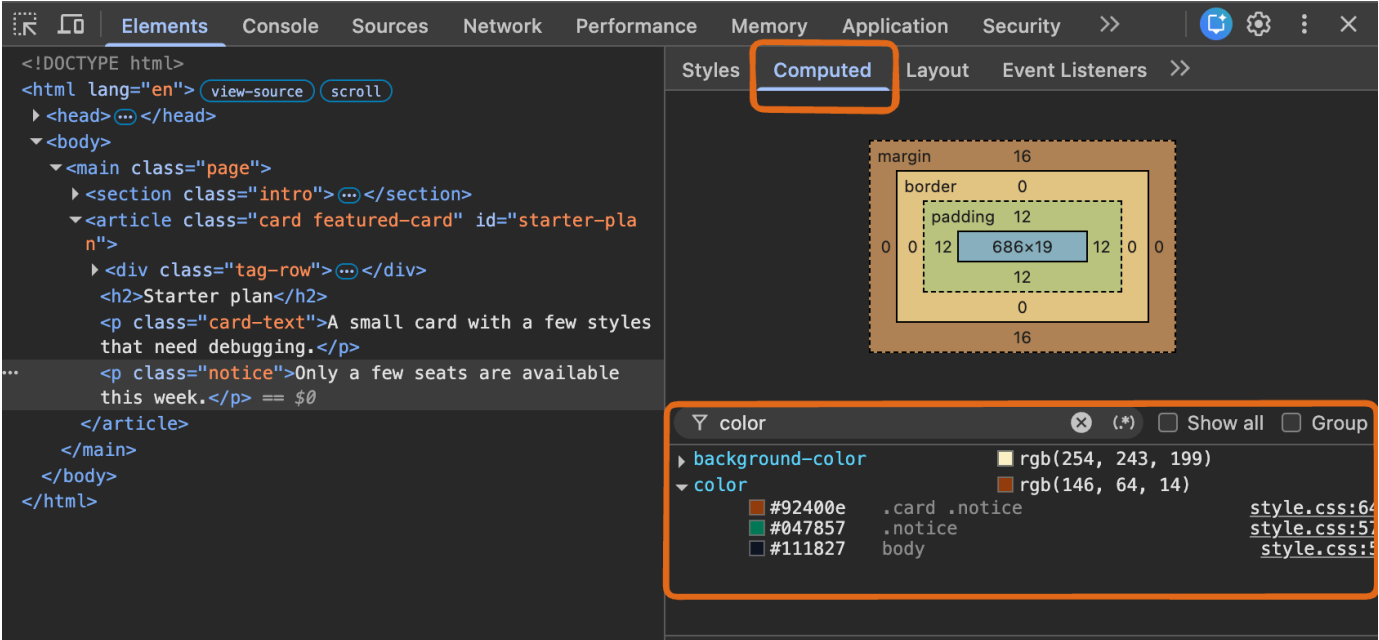
Use Computed for the final answer

The `Styles` panel shows the rules that matched. The `Computed` panel shows the final value the browser is using.

For example, inspect the notice and search for `color` in the `Computed` panel.

You should see the final text color value from `.card .notice` . Some browsers may show it as `#92400e` ; others may show the same color as `rgb(146, 64, 14)` .

If DevTools shows a small arrow or expandable row next to the computed value, open it. It can often show which CSS rule supplied that final value.



This is useful when a page has many stylesheets. You do not have to read every file first. Start with the element, find the property, and let DevTools show the winner.

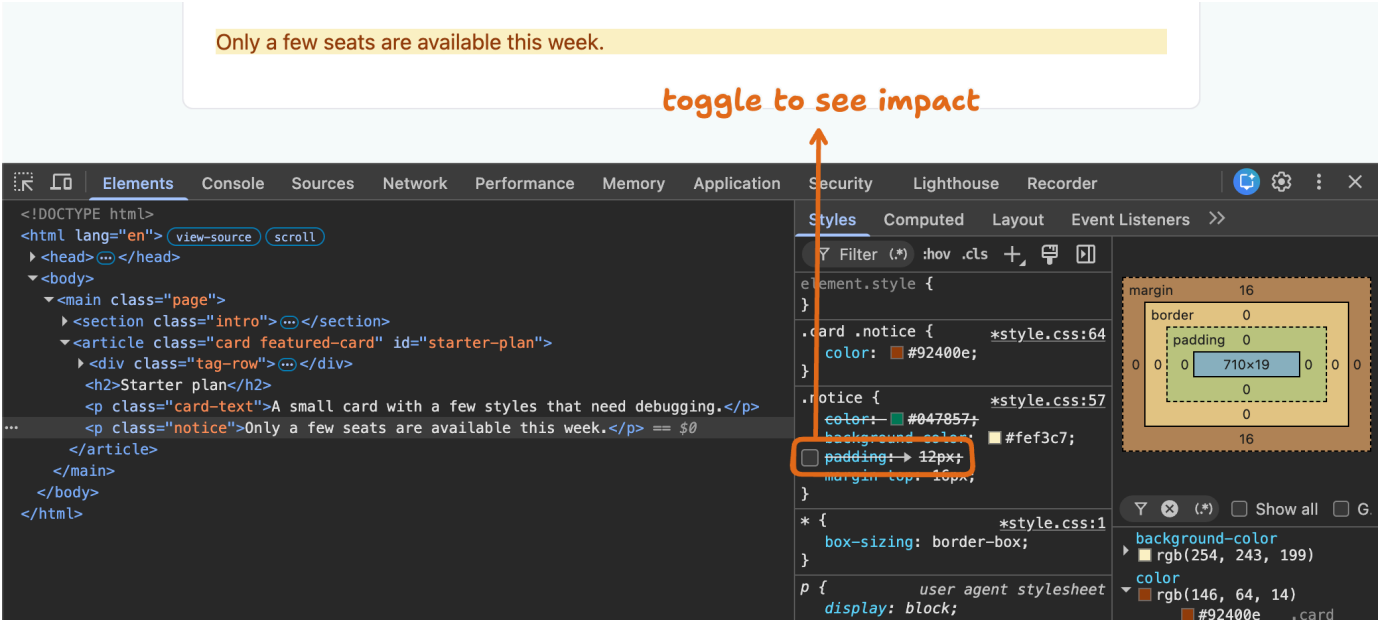
Toggle declarations on and off

Most DevTools let you turn a declaration off with a checkbox.

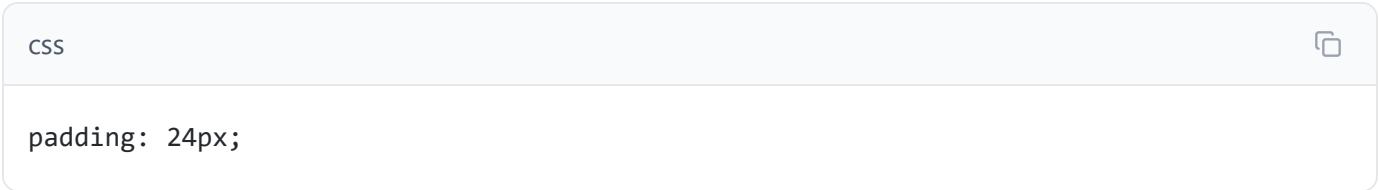
Try the toggle with this small sequence:

1. Inspect the notice.
2. Find `color: #92400e;` in `.card .notice` .
3. Turn that declaration off.
4. Watch the notice text change to the `.notice` color.

This is a quick way to test cause and effect. You are asking the browser, "What happens if this declaration is not here?"



You can also click values and edit them:



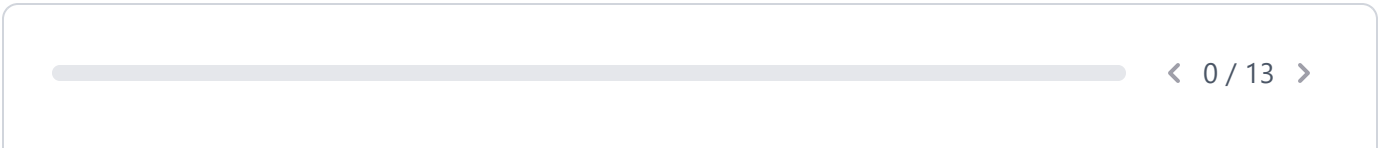
Try changing it to `16px` , `32px` , or `40px` in DevTools. Then put the value you like into `style.css` .

Try it

Before moving on, recreate the same debugging clues in a few new ways:

- Change the HTML class from `featured-card` to `featured-plan` , but do not change the CSS yet. Inspect the card and confirm the `.featured-card` rule no longer appears as a match.
- Change `.card .notice` to `.notice` and reload. Predict which notice color wins, then inspect the notice to confirm.
- Remove `display: flex` from `.tag-row` . Inspect the tag row and notice that `gap` no longer creates visible space.
- Inspect `.card` , change its padding in DevTools, and watch the box model diagram update.
- In DevTools, turn off `background-color: #fef3c7;` on `.notice` . Notice that the text remains, but the warning box no longer feels like a message.
- Add an invalid value like `color: brightblue;` to `.lead` in DevTools. Watch how the browser ignores invalid CSS.

The goal is not to memorize every DevTools panel. The goal is to build a habit: inspect the element, find the property, and look for the clue.





Knew

0 Items



Learnt

0 Items



Skipped

0 Items



ResetProgress

Test Your Understanding

What should you do first when a CSS style surprises you?

[Click to Reveal the Answer](#)



Already Know that



Didn't Know that



Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →